

Section 96 Editorial — Master Continuous Geometry Mixed Contest

A. Safe Operating Region

How to find it

Sort directed boundaries by angle, discard weaker parallel copies, and maintain feasible consecutive intersections in a deque.

Correctness

The deque invariant retains exactly the processed region boundaries in angular order; every removed corner violates the new constraint and can never return. Closing yields precisely the intersection polygon.

Complexity

$O(n \log n)$ time and $O(n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
    P operator-(P b)const {
        return {
            x-b.x,y-b.y
        }
    };
    P operator*(double k)const {
        return {
            x*k,y*k
        }
    };
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
```

```

}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
vector<P>halfplanes(vector<pair<P,P>>raw) {
    vector<L>a;
    for(auto [p,q]:raw) {
        P v=q-p;
        a.push_back({p,v,atan2(v.y,v.x)});
    }
    sort(a.begin(),a.end(),[](L x,L y){return x.angle<y.angle;});
    vector<L>u;
    for(L z:a) {
        if(!u.empty()&&abs(cross(u.back().v,z.v))<=EPS&&dot(u.back().v,z.v)>0) {
            if(cross(u.back().v,z.p-u.back().p)>EPS)u.back()=z;
        }
        else u.push_back(z);
    }
    deque<L>q;
    deque<P>pts;
    auto inside=[](L l,P p) {
        return cross(l.v,p-l.p)>=-EPS;
    }
    ;
    for(L z:u) {
        while(!pts.empty()&&!inside(z,pts.back()))pts.pop_back(),q.pop_back();
        while(!pts.empty()&&!inside(z,pts.front()))pts.pop_front(),q.pop_front();
        if(!q.empty()&&abs(cross(q.back().v,z.v))<=EPS)return {
        }
        ;
        if(!q.empty())pts.push_back(meet(q.back(),z));
        q.push_back(z);
    }
    while(!pts.empty()&&!inside(q.front(),pts.back()))pts.pop_back(),q.pop_back();
    while(!pts.empty()&&!inside(q.back(),pts.front()))pts.pop_front(),q.pop_front();
    if(q.size()<3)return {
    }
    ;
    pts.push_back(meet(q.back(),q.front()));
    return vector<P>(pts.begin(),pts.end());
}
double area(vector<P>p) {
    double s=0;
    for(int i=0;i<(int)p.size();i++)s+=cross(p[i],p[(i+1)%p.size()]);
    return abs(s)/2;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<pair<P,P>>l(n);
    for(auto&[a,b]:l)cin>>a.x>>a.y>>b.x>>b.y;
    cout<<area(halfplanes(l))<<'\\n';
}

```

Python

```
import sys, math, random
from collections import deque
EPS = 1e-10

def add(a, b):
    return (a[0] + b[0], a[1] + b[1])

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def mul(a, k):
    return (a[0] * k, a[1] * k)

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def intersection(a, b):
    t = cross(sub(b[0], a[0]), b[1]) / cross(a[1], b[1])
    return add(a[0], mul(a[1], t))

def halfplanes(raw):
    lines = []
    for a, b in raw:
        v = sub(b, a)
        lines.append((a, v, math.atan2(v[1], v[0])))
    lines.sort(key=lambda z: z[2])
    unique = []
    for p, v, angle in lines:
        if unique and abs(cross(unique[-1][1], v)) <= EPS and (dot(unique[-1][1], v) > 0):
            if cross(unique[-1][1], sub(p, unique[-1][0])) > EPS:
                unique[-1] = (p, v, angle)
            else:
                unique.append((p, v, angle))
    dq = deque()
    points = deque()

    def inside(line, p):
        return cross(line[1], sub(p, line[0])) >= -EPS
    for line in unique:
        while points and (not inside(line, points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(line, points[0])):
            points.popleft()
            dq.popleft()
        if dq and abs(cross(dq[-1][1], line[1])) <= EPS:
            return []
        if dq:
            points.append(intersection(dq[-1], line))
            dq.append(line)
        while points and (not inside(dq[0], points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(dq[-1], points[0])):
            points.popleft()
            dq.popleft()
    if len(dq) < 3:
        return []
```

```

    points.append(intersection(dq[-1], dq[0]))
    return list(points)

def area(poly):
    return abs(sum((cross(poly[i], poly[(i + 1) % len(poly)])) for i in range(len(poly))))) / 2 if len(poly) >= 3 else 0.0

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    raw = []
    for i in range(n):
        x1, y1, x2, y2 = d[1 + 4 * i:5 + 4 * i]
        raw.append((x1, y1), (x2, y2))
    print(f'{area(halfplanes(raw)):.10f}')
if __name__ == '__main__':
    main()

```

B. Round Table Clearance

How to find it

Offset every counterclockwise edge inward by radius r ; test whether the shifted half-planes intersect, then binary-search r .

Correctness

A center supports radius r exactly when its distance from every edge is at least r , which is exactly membership in all shifted half-planes. Feasibility is monotone.

Complexity

$O(n \log n \log \text{precision})$ with general HPI.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
}
;

```

```

double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
vector<P>halfplanes(vector<pair<P,P>>raw) {
    vector<L>a;
    for(auto[p,q]:raw) {
        P v=q-p;
        a.push_back({p,v,atan2(v.y,v.x)});
    }
    sort(a.begin(),a.end(),[](L x,L y){return x.angle<y.angle;});
    vector<L>u;
    for(L z:a) {
        if(!u.empty()&&abs(cross(u.back().v,z.v))<=EPS&&dot(u.back().v,z.v)>0) {
            if(cross(u.back().v,z.p-u.back().p)>EPS)u.back().v=z;
        }
        else u.push_back(z);
    }
    deque<L>q;
    deque<P>pts;
    auto inside=[](L l,P p) {
        return cross(l.v,p-l.p)>=-EPS;
    };
    for(L z:u) {
        while(!pts.empty()&&!inside(z,pts.back()))pts.pop_back(),q.pop_back();
        while(!pts.empty()&&!inside(z,pts.front()))pts.pop_front(),q.pop_front();
        if(!q.empty()&&abs(cross(q.back().v,z.v))<=EPS)return {
        }
        ;
        if(!q.empty())pts.push_back(meet(q.back(),z));
        q.push_back(z);
    }
    while(!pts.empty()&&!inside(q.front(),pts.back()))pts.pop_back(),q.pop_back();
    while(!pts.empty()&&!inside(q.back(),pts.front()))pts.pop_front(),q.pop_front();
    if(q.size()<3)return {
    }
    ;
    pts.push_back(meet(q.back(),q.front()));
    return vector<P>(pts.begin(),pts.end());
}
double area(vector<P>p) {
    double s=0;
    for(int i=0;i<(int)p.size();i++)s+=cross(p[i],p[(i+1)%p.size()]);
    return abs(s)/2;
}
vector<P>shifted(vector<P>p,double r) {
    vector<pair<P,P>>l;
    for(int i=0;i<(int)p.size();i++) {

```

```

        P q=p[(i+1)%p.size()],v=q-p[i],s= {
            -v.y*r/norm(v),v.x*r/norm(v)
        }
        ;
        l.push_back({p[i]+s,q+s});
    }
    return halfplanes(l);
}

double inradius(vector<P>p) {
    double lo=0,hi=1;
    for(P x:p)for(P y:p)hi=max(hi,norm(x-y));
    for(int z=0;z<70;z++) {
        double m=(lo+hi)/2;
        (shifted(p,m).empty()?hi:lo)=m;
    }
    return lo;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<inradius(p)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def add(a, b):
    return (a[0] + b[0], a[1] + b[1])

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def mul(a, k):
    return (a[0] * k, a[1] * k)

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def intersection(a, b):
    t = cross(sub(b[0], a[0]), b[1]) / cross(a[1], b[1])
    return add(a[0], mul(a[1], t))

def halfplanes(raw):
    lines = []
    for a, b in raw:
        v = sub(b, a)
        lines.append((a, v, math.atan2(v[1], v[0])))
    lines.sort(key=lambda z: z[2])

```

```

unique = []
for p, v, angle in lines:
    if unique and abs(cross(unique[-1][1], v)) <= EPS and (dot(unique[-1][1], v) > 0):
        if cross(unique[-1][1], sub(p, unique[-1][0])) > EPS:
            unique[-1] = (p, v, angle)
        else:
            unique.append((p, v, angle))
dq = deque()
points = deque()

def inside(line, p):
    return cross(line[1], sub(p, line[0])) >= -EPS
for line in unique:
    while points and (not inside(line, points[-1])):
        points.pop()
        dq.pop()
    while points and (not inside(line, points[0])):
        points.popleft()
        dq.popleft()
    if dq and abs(cross(dq[-1][1], line[1])) <= EPS:
        return []
    if dq:
        points.append(intersection(dq[-1], line))
        dq.append(line)
    while points and (not inside(dq[0], points[-1])):
        points.pop()
        dq.pop()
    while points and (not inside(dq[-1], points[0])):
        points.popleft()
        dq.popleft()
    if len(dq) < 3:
        return []
    points.append(intersection(dq[-1], dq[0]))
    return list(points)

def shifted_polygon(poly, r):
    lines = []
    for i, p in enumerate(poly):
        q = poly[(i + 1) % len(poly)]
        v = sub(q, p)
        length = norm(v)
        shift = (-v[1] * r / length, v[0] * r / length)
        lines.append((add(p, shift), add(q, shift)))
    return halfplanes(lines)

def inradius(poly):
    low = 0.0
    high = max(max((x for x, y in poly)) - min((x for x, y in poly)), max((y for x, y in poly)) - min((y for x, y in poly)))
    for _ in range(70):
        mid = (low + high) / 2
        if shifted_polygon(poly, mid):
            low = mid
        else:
            high = mid
    return low

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{inradius(p):.10f}')
if __name__ == '__main__':
    main()

```

C. Sensor Overlap

How to find it

Separate disjoint and containment cases before using the two-sector lens formula.

Correctness

In the proper-overlap case the common region partitions into two sectors minus their center triangles; the other classifications are immediate.

Complexity

$O(1)$ time and memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    }
};
P operator*(double k)const {
    return {
        x*k,y*k
    }
};
};
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
```

```

struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    C a,b;
    cin>>a.p.x>>a.p.y>>a.r>>b.p.x>>b.p.y>>b.r;
    cout<<overlap(a,b)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def overlap(c1, c2):
    x1, y1, r1 = c1
    x2, y2, r2 = c2
    d = math.hypot(x1 - x2, y1 - y2)
    if d >= r1 + r2:
        return 0.0
    if d <= abs(r1 - r2):
        return math.pi * min(r1, r2) ** 2
    a = math.acos(max(-1, min(1, (d * d + r1 * r1 - r2 * r2) / (2 * d * r1))))
    b = math.acos(max(-1, min(1, (d * d + r2 * r2 - r1 * r1) / (2 * d * r2))))
    return r1 * r1 * a + r2 * r2 * b - d * r1 * math.sin(a)

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    print(f'overlap(d[:3], d[3:6]):{.10f}')
if __name__ == '__main__':
    main()

```

D. Emergency Broadcast Disk

How to find it

Randomize insertion order and rebuild only when a point violates the current circle, fixing one, two, then three boundary witnesses.

Correctness

At every nesting level the violating fixed points must lie on the new optimum boundary; one to three such points uniquely determine the minimum circle for the processed prefix.

Complexity

Expected $O(n)$ time and $O(n)$ storage.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        };
    }
    P operator-(P b)const {
        return {
            x-b.x,y-b.y
        };
    }
    P operator*(double k)const {
        return {
            x*k,y*k
        };
    }
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
};
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
};
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
```

```

    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}

int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}

C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    };
}

C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    };
    return {
        o,norm(o-a)
    };
}

C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        },-1
    };
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    };
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        };
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;

```

```

vector<P>p(n);
for(auto&x:p)cin>>x.x>>x.y;
C =mec(p);
cout<<c.p.x<<' '<<c.p.y<<' '<<c.r<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle2(a, b):
    c = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
    return (c, norm(sub(a, b)) / 2)

def circle3(a, b, c):
    d = 2 * cross(sub(b, a), sub(c, a))
    ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
    uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
    center = (ux, uy)
    return (center, norm(sub(center, a)))

def minimum_circle(points):
    p = points[:]
    random.Random(947311).shuffle(p)
    circle = ((0.0, 0.0), -1.0)

    def outside(x, c):
        return c[1] < 0 or norm(sub(x, c[0])) > c[1] + 1e-09

    for i in range(len(p)):
        if outside(p[i], circle):
            circle = (p[i], 0.0)
            for j in range(i):
                if outside(p[j], circle):
                    circle = circle2(p[i], p[j])
                    for k in range(j):
                        if outside(p[k], circle):
                            circle = circle3(p[i], p[j], p[k])

    return circle

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    c, r = minimum_circle(p)
    print(f'{{c[0]:.10f}} {{c[1]:.10f}} {{r:.10f}}')
if __name__ == '__main__':
    main()

```

E. Empty Observation Zone

How to find it

Enumerate the final Delaunay triangles and maximize their circumradii.

Correctness

A finite empty circle through three sites is a Delaunay face circumcircle, and every Delaunay face supplies such an empty circle.

Complexity

Expected $O(n^2)$ here.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b) const {
        return {
            x+b.x,y+b.y
        };
    }
    P operator-(P b) const {
        return {
            x-b.x,y-b.y
        };
    }
    P operator*(double k) const {
        return {
            x*k,y*k
        };
    }
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
};
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
```

```

    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    }
;
}
C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    }
;
    return {
        o,norm(o-a)
    }
;
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        }
        ,-1
    }
;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        }
;
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
}

```

```

struct T {
    int a,b,c;
}
;
double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}
bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}
pair<vector<T>,vector<pair<int,int>>> delaunay(vector<P>p) {
    int n=p.size();
    if(n<3) return {
        {
        },
        {
        }
    }
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p) minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
    p.push_back({cx,cy+10000*s});
    vector<T>tri= {
        {
            n,n+1,n+2
        }
    }
    ;
    for(int x=0;x<n;x++) {
        map<pair<int,int>,pair<int,pair<int,int>>>edges;
        vector<char>bad(tri.size());
        for(int i=0;i<(int)tri.size();i++) if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
            bad[i]=1;
            for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
                auto key=minmax(e.first,e.second);
                edges[key].first++;
                edges[key].second=e;
            }
        }
        vector<T>next;
        for(int i=0;i<(int)tri.size();i++) if(!bad[i]) next.push_back(tri[i]);
        for(auto [key,data]:edges) if(data.first==1) {
            auto [u,v]=data.second;
            if(orient(p[u],p[v],p[x])<0) swap(u,v);
            next.push_back({u,v,x});
        }
        tri.swap(next);
    }
    vector<T>clean;
    set<pair<int,int>>edges;
    for(T t:tri) if(max({t.a,t.b,t.c})<n) {
        clean.push_back(t);
        edges.insert(minmax(t.a,t.b));
        edges.insert(minmax(t.b,t.c));
        edges.insert(minmax(t.c,t.a));
    }
    return {
        clean,vector<pair<int,int>>(edges.begin(),edges.end())
    }
}

```

```

    }
    ;
}
double radii(vector<P>p,bool maximum) {
    auto[tri,e]=delaunay(p);
    double ans=0;
    for(T t:tri) {
        double r=circle3(p[t.a],p[t.b],p[t.c]).r;
        if(maximum)ans=max(ans,r);
        else ans+=r;
    }
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<radii(p,true)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle3(a, b, c):
    d = 2 * cross(sub(b, a), sub(c, a))
    ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
    uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
    center = (ux, uy)
    return (center, norm(sub(center, a)))

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]
    cx = c[0] - p[0]
    cy = c[1] - p[1]
    det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) * (ax * by - ay * bx)
    return det > EPS

```

```

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]
    tri = [(n, n + 1, n + 2)]
    for idx in range(n):
        bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
        count = {}
        direct = {}
        for t in bad:
            for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
                key = tuple(sorted(e))
                count[key] = count.get(key, 0) + 1
                direct[key] = e
        gone = set(bad)
        tri = [t for t in tri if t not in gone]
        for key, cnt in count.items():
            if cnt == 1:
                u, v = direct[key]
                if orient(p[u], p[v], p[idx]) < 0:
                    u, v = (v, u)
                tri.append((u, v, idx))
    tri = [t for t in tri if max(t) < n]
    edges = set()
    for a, b, c in tri:
        edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
    return (tri, sorted(edges))

def radius_sum(points, maximum=False):
    tri, _ = delaunay(points)
    values = [circle3(points[a], points[b], points[c])[1] for a, b, c in tri]
    return (max(values) if values else 0.0) if maximum else sum(values)

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{radius_sum(p, True):.10f}')
if __name__ == '__main__':
    main()

```

F. Low-Cost Fiber

How to find it

Build Delaunay edges and run Kruskal on that sparse graph.

Correctness

Every Euclidean MST edge has an empty-disk certificate and belongs to Delaunay, so discarding all other complete-graph edges preserves an optimum.

Complexity

Expected $O(n^2 + n \log n)$ here.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        };
    }
    P operator-(P b)const {
        return {
            x-b.x,y-b.y
        };
    }
    P operator*(double k)const {
        return {
            x*k,y*k
        };
    }
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
};
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
};
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
```

```

    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}

C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    }
};

C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    }
    ;
    return {
        o,norm(o-a)
    }
    ;
}

C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        },-1
    }
    ;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
    ;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        }
        ;
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}

struct T {
    int a,b,c;
}
;

double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}

bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}

```

```

pair<vector<T>,vector<pair<int,int>>> delaunay(vector<P>p) {
    int n=p.size();
    if(n<3) return {
        {
        },
        {
        }
    }
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p) minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
    p.push_back({cx,cy+10000*s});
    vector<T>tri= {
        {
            n,n+1,n+2
        }
    }
    ;
    for(int x=0;x<n;x++) {
        map<pair<int,int>,pair<int,pair<int,int>>>edges;
        vector<char>bad(tri.size());
        for(int i=0;i<(int)tri.size();i++) if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
            bad[i]=1;
            for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
                auto key=minmax(e.first,e.second);
                edges[key].first++;
                edges[key].second=e;
            }
        }
        vector<T>next;
        for(int i=0;i<(int)tri.size();i++) if(!bad[i]) next.push_back(tri[i]);
        for(auto [key,data]:edges) if(data.first==1) {
            auto [u,v]=data.second;
            if(orient(p[u],p[v],p[x])<0) swap(u,v);
            next.push_back({u,v,x});
        }
        tri.swap(next);
    }
    vector<T>clean;
    set<pair<int,int>>edges;
    for(T t:tri) if(max({t.a,t.b,t.c})<n) {
        clean.push_back(t);
        edges.insert(minmax(t.a,t.b));
        edges.insert(minmax(t.b,t.c));
        edges.insert(minmax(t.c,t.a));
    }
    return {
        clean,vector<pair<int,int>>(edges.begin(),edges.end())
    }
    ;
}

double radii(vector<P>p,bool maximum) {
    auto [tri,e]=delaunay(p);
    double ans=0;
    for(T t:tri) {
        double r=circle3(p[t.a],p[t.b],p[t.c]).r;
        if(maximum) ans=max(ans,r);
        else ans+=r;
    }
    return ans;
}

```

```

}
struct DSU {
    vector<int>p;
    DSU(int n):p(n) {
        iota(p.begin(),p.end(),0);
    }
    int f(int x) {
        return p[x]==x?x:p[x]=f(p[x]);
    }
}
;
double mst(vector<P>p) {
    int n=p.size();
    if(n<2)return 0;
    auto[tri,e]=delaunay(p);
    if(e.empty())for(int i=1;i<n;i++)e.push_back({i-1,i});
    vector<tuple<double,int,int>>all;
    for(auto[u,v]:e)all.push_back({norm(p[u]-p[v]),u,v});
    sort(all.begin(),all.end());
    DSU d(n);
    double ans=0;
    for(auto[w,u,v]:all) {
        u=d.f(u);
        v=d.f(v);
        if(u!=v)d.p[u]=v,ans+=w;
    }
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<mst(p)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def norm(a):
    return math.hypot(a[0], a[1])

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]

```

```

cx = c[0] - p[0]
cy = c[1] - p[1]
det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) *
return det > EPS

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]
    tri = [(n, n + 1, n + 2)]
    for idx in range(n):
        bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
        count = {}
        direct = {}
        for t in bad:
            for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
                key = tuple(sorted(e))
                count[key] = count.get(key, 0) + 1
                direct[key] = e
        gone = set(bad)
        tri = [t for t in tri if t not in gone]
        for key, cnt in count.items():
            if cnt == 1:
                u, v = direct[key]
                if orient(p[u], p[v], p[idx]) < 0:
                    u, v = (v, u)
                tri.append((u, v, idx))
        tri = [t for t in tri if max(t) < n]
        edges = set()
        for a, b, c in tri:
            edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
        return (tri, sorted(edges))

def mst(points):
    n = len(points)
    if n < 2:
        return 0.0
    _, edges = delaunay(points)
    if not edges:
        edges = [(i, i + 1) for i in range(n - 1)]
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    ans = 0.0
    for w, u, v in sorted(((norm(sub(points[u], points[v])), u, v) for u, v in edges)):
        u = find(u)
        v = find(v)
        if u != v:
            parent[u] = v
            ans += w
    return ans

```

```
def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{mst(p):.10f}')
if __name__ == '__main__':
    main()
```

Mixed-contest verification

For A and B, independently clip a large bounding polygon on tiny cases and check the monotonicity of every tested radius. For C, compare the result with the appropriate limiting cases: zero at external separation and the smaller disk's area at containment. For D, enumerate all support circles. For E, check empty circumcircles. For F, compare with complete-graph Kruskal.

Apply translations, rotations, input permutations, and uniform scaling. The combinatorial answer should be invariant, while lengths scale once and areas twice. These metamorphic tests expose inconsistent orientation and tolerance policies that ordinary random points rarely trigger.